PAGE 1 · OVERVIEW

BusGo — Bus Booking Demo

A clean, full-stack bus-ticket booking web application built to demonstrate how a modern web app works end to end — search, seat selection, payment and ticketing — without the complexity of a production system.

What it is

BusGo lets a user search bus routes between cities, pick a date, choose seats on a live seat map, enter passenger details, "pay" with a mock card, and receive a confirmed ticket with a PNR. Past bookings are saved and can be viewed later.

Purpose & scope

This is a **demonstration / learning project**. The goal is code that is easy to read and explain, while the interface still looks production-grade. It deliberately avoids a real database, real user accounts, and a real payment gateway.

The whole idea in one breath

API routes are the *backend*. `fetch` talks to the backend. `useState` remembers things on a page. One *Booking Context* holds the booking-in-progress. *Material UI* draws the interface.

At a glance

Next.js (full-stack) · JavaScript · React · Material UI · React Context · JSON-file storage

Key characteristics

- **One codebase, one app.** The frontend and the backend live together in a single Next.js project.
- **Plain JavaScript** (no TypeScript) to keep the code approachable.
- **Server-side persistence** using a simple JSON file — bookings and taken seats survive page reloads and even server restarts.
- **Responsive & sharp UI** that works on phones and desktops.

What the app can do

The user journey runs across five screens, each adding one step of the booking.

1 · Search

Pick a **From** city, a **To** city and a **date of journey**. Cities are suggested as you type, and there is a one-tap swap button.

2 · Results

See all buses for the route with operator, timings, duration, price, rating and seats left. **Filter** by type (AC / Sleeper / Seater), departure window and price, and **sort** by price, time, duration or rating.

3 · Seat selection

An interactive **seat map** with hover/selection animation. Booked seats are greyed out; sleeper buses show separate **lower & upper decks**.

4 · Passenger details

Enter a name, age and gender per seat, plus contact details and **boarding / dropping points**.

5 · Payment (mock)

A realistic **card-flip animation**. A "Fill test card" shortcut speeds up demos. No real money moves.

6 · Confirmation

A printable ticket with a unique **PNR**, fired off with a **confetti** celebration. The ticket is saved automatically.

Always-available features

My Bookings

Every confirmed ticket is listed and can be re-opened or printed. This proves the data really persists on the server.

Reset demo

A single button wipes all bookings and freed seats back to the starting state — perfect before a fresh demo.

Quality touches

- **Simulated network latency** shows polished loading skeletons, so it feels like a real online service.
- **Live seat availability** — once a seat is booked it cannot be booked again (the server rejects double-booking).
- **Responsive design** — the navigation collapses to icons on phones and every screen reflows cleanly.
- **Accessible seat buttons**, snackbars for feedback, and helpful empty states.

Backend — language & building blocks

Language & runtime

The backend is written in **JavaScript** and runs on **Node.js** (the JavaScript runtime that executes code on the server instead of in the browser).

Framework — Next.js Route Handlers

There is no separate server project. The backend is a set of **API Route Handlers** inside the same Next.js app — each file under `app/api/…/route.js` exports `GET` / `POST` functions that receive a request and return a JSON response.

Data storage — no database

To stay simple, BusGo uses two data sources instead of a database:

- **Seed data in code** (`lib/seed/`) — the list of cities and a handful of reusable bus "templates". The same templates are applied to any route the user searches, so every search returns results.
- **A JSON file** (`.data/bookings.json`) — written with Node's built-in `fs` module. It stores confirmed bookings and which seats are taken, so data survives reloads and restarts.

What the backend does

Capability	How it works
Search buses	Builds the bus list for a route and reports how many seats are still free.
Seat availability	Taken seats = seats marked booked in the seed data *plus* seats from real bookings, keyed by bus + date.
Create booking	Validates the chosen seats are still free, then saves the booking and assigns a random **PNR**. Rejects double-booking with an error.
List / fetch bookings	Returns all saved bookings, or a single booking by id (used by the ticket page).
Reset	Clears the JSON file back to the empty seed state.

Deliberately left out

No SQL database · No login / auth · No real payment gateway — these would be the next steps to make it production-grade, but are unnecessary for a demo.

Frontend — libraries we use

The interface is built with **React** (rendered by Next.js) and styled with **Material UI**. Shared state is handled by React's built-in **Context** — intentionally *not* Redux, to keep things simple to explain.

Library	What it is for
`next`	The framework: file-based routing, the dev server, the production build, and the API routes.
`react` / `react-dom`	The core UI library — builds the screen out of reusable components.
`@mui/material`	Material UI — ready-made, polished components (buttons, inputs, cards, stepper, slider…).
`@mui/icons-material`	The icon set (bus, seat, search, ticket, etc.).
`@emotion/react`, `@emotion/styled`	The styling engine Material UI uses under the hood.
`@mui/material-nextjs`	Glue that makes Material UI styling work correctly with the Next.js App Router.
`@mui/x-date-pickers`	The calendar / date-picker used for the journey date.
`dayjs`	A tiny date library used to format and calculate times.
`canvas-confetti`	The confetti burst on the confirmation page.
`next/font` (Inter)	Loads the "Inter" font cleanly and quickly.

How state is organised

Local state — `useState`

Each screen remembers its own temporary values (form fields, the active step, loading flags).

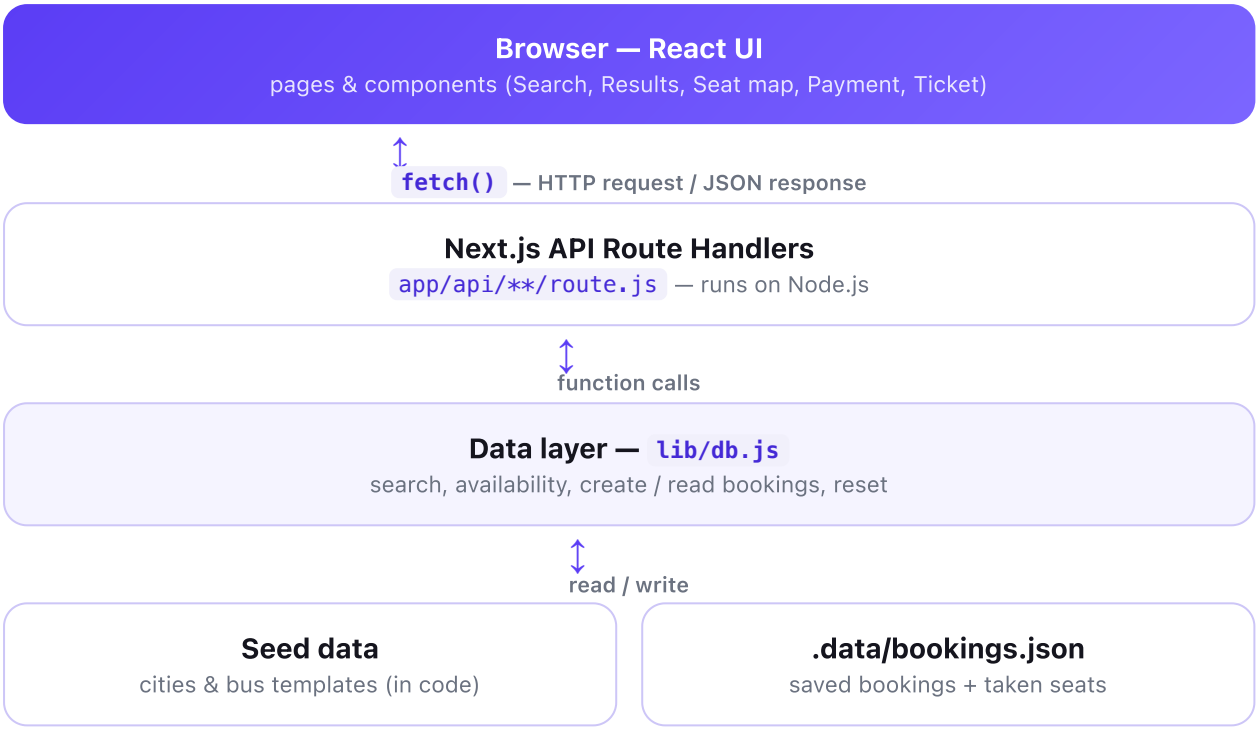
Shared state — `BookingContext`

One small Context holds the "booking in progress" (search, selected bus, seats, passengers) so it can be shared as the user moves between pages.

Routing & data

- **Routing:** Next.js maps folders to URLs (`app/results/page.js` → `/results`). Navigation uses `next/navigation` .
- **Data:** pages call the backend with the browser's built-in `fetch` and receive JSON.

How the frontend talks to the backend



API endpoints

Method	Path	What it does
GET	`/api/cities`	List of cities for the search boxes.
GET	`/api/buses?from&to&date`	All buses for a route, each with seats-available.
GET	`/api/buses/[busId]?date`	One bus in detail: seat layout + which seats are booked.
POST	`/api/bookings`	Create a booking. Body: `busId, date, seats, passengers, contact`. Returns the booking with a **PNR**; rejects if seats were taken.
GET	`/api/bookings`	All saved bookings (the "My Bookings" list).
GET	`/api/bookings/[id]`	A single booking — used by the confirmation / ticket page.
POST	`/api/reset`	Clear all bookings back to the seed state (demo helper).

A typical request

`GET /api/buses?from=Bengaluru&to=Goa&date=2026-07-02` → returns a JSON array of buses. The Results page renders one card per item. Selecting a seat and paying sends `POST /api/bookings`, and the server replies with a confirmed ticket object.

Glossary — the key terms

API — Application Programming Interface. A defined set of URLs the frontend can call to ask the backend to do something or return data.

HTTP request / response — the message a browser sends to a server (e.g. "GET this data") and the reply it gets back.

Node.js — a runtime that lets JavaScript run on a server (outside the browser). Our backend runs on it.

Next.js — a React framework that provides routing, the build system, and a place to write backend API code — all in one project.

App Router — Next.js's system where the folder structure under `app/` defines the pages and URLs.

API Route Handler — a backend function (`route.js`) that answers an API request and returns JSON.

React — a library for building user interfaces out of reusable "components".

Component — a self-contained piece of UI (e.g. a BusCard or SeatMap) that can be reused.

JSX — the HTML-like syntax used inside React components to describe what to show.

State / `useState` — values a component remembers and re-renders when they change (e.g. selected seats).

React Context — a way to share one piece of state across many components without passing it down manually. BusGo uses it for the booking-in-progress.

Props — the inputs you pass into a component to configure it.

Material UI (MUI) — a library of pre-built, good-looking React components and an icon set.

`fetch` — the browser's built-in function for calling an API over HTTP.

JSON — JavaScript Object Notation; the simple text format used to send data between frontend and backend.

PNR — Passenger Name Record; the short booking reference code printed on the ticket.

localStorage vs server — localStorage saves data inside one browser. BusGo instead saves on the *server* (a JSON file), so bookings are shared across reloads and devices.

Putting it together

A **React** component shows the UI and keeps **state**. When it needs data it uses **fetch** to call an **API** route handler running on **Node.js** inside **Next.js**. The handler reads or writes data and replies with **JSON**. Shared booking data lives in **React Context**, and **Material UI** makes it all look polished.